

Clinic Booking

A DEVOPS PROJECT REPORT SUBMITTED TO

THE NATIONAL INSTITUTE OF ENGINEERING, MYSURU
(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)



in partial fulfillment for the award of degree of

Bachelor of Engineering in Computer Science and Engineering

Submitted By

Pallavi SE 4NI24CS421
Syeda Mizba 4NI24CS427

Under the guidance of

Mr. Joyan Prajwal Alvares
Assistant Professor
Department of CS&E
NIE, Mysuru



Department of Computer Science & Engineering
THE NATIONAL INSTITUTE OF ENGINEERING

(Autonomous Institution)

Mysuru - 570 008 2025-



THE NATIONAL INSTITUTE OF ENGINEERING

(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)



Department of Computer Science & Engineering

CERTIFICATE

This is to certify that the project work entitled Clinic Booking is a Bonafide work carried out by **Syeda Mizba (4NI24CS427)**, **Pallavi SE (4NI24CS421)** in partial fulfillment for the award of degree of **Bachelor of Engineering in Computer Science and Engineering**, of Visvesvaraya Technological University, Belagavi, during the year **2025-26**. It is certified that all corrections / suggestions indicated during internal assessment have been incorporated and the corrected copy has been deposited in the department library. This project report has been approved in partial fulfillment for the award of the said degree as per academic regulations of The National Institute of Engineering (Autonomous Institution).

Mr. Joyan Prajwal Alvares

Assistant Professor

Dept. of CS&E

NIE, Mysuru

Dr. Anitha R

Professor and Head

Dept. of CS&E

NIE, Mysuru

ABSTRACT

The Clinic Booking Website project is a web-based application developed and deployed using modern DevOps tools and practices. The primary objective of this project is to automate the software build and deployment process through a Continuous Integration and Continuous Deployment (CI/CD) pipeline. The application source code is managed using GitHub, containerized using Docker, and automatically deployed using Jenkins.

The website provides a user-friendly interface for booking clinic appointments and demonstrates the implementation of automation in software delivery. Docker ensures platform-independent deployment by packaging the application into lightweight containers, while Jenkins automates the workflow of building, testing, and deploying the application whenever changes are pushed to the GitHub repository.

This project highlights the importance of DevOps practices in improving deployment speed, reducing manual errors, and ensuring consistency across environments. The successful implementation of the CI/CD pipeline demonstrates how automation tools can simplify application deployment and maintenance in real-world software development environments.

Docker is used to containerise both the FastAPI application and MongoDB database, enabling the project to run consistently across different environments without dependency issues. Jenkins is integrated to implement Continuous Integration and Continuous Deployment (CI/CD), where code changes pushed to GitHub automatically trigger test execution via pytest, ensuring build reliability.

GitHub is used for source code management and version control, Docker is used for containerizing the application, and Jenkins automates the build and deployment process. By integrating these tools, the project demonstrates how DevOps practices improve software deployment efficiency and reliability.

ACKNOWLEDGMENT

I sincerely owe my gratitude to all the persons who helped and guided me in completing this mini project.

I am thankful to **Dr. Nagendra Parashar**, Principal, NIE College, Mysuru, for all the support she has rendered.

I thank **Dr. Anitha R**, Professor and Head, Department of Computer Science and Engineering, for her constant support and encouragement throughout the tenure for the mini project work.

I would like to sincerely thank my guide **Mr. Joyan Prajwal Alvares**, Assistant Professor, Department of Computer Science and Engineering, for providing relevant information, valuable guidance and encouragement to complete this minor project.

Syeda Mizba	4NI24CS427
Pallavi SE	4NI23CS421

TABLE OF CONTENTS

No.	CONTENTS	PAGE NO.
	Abstract	I
	Acknowledgement	II
	Table of Contents	III
	List of Figures	IV
1.	Introduction	1–2
1.1	Background	
1.2	Purpose	
1.3	Objectives	
2.	Literature Review	3–5
3.	Existing System and Proposed System	6–10
3.1	Existing System	
3.2	Proposed System	
4.	System Requirements	
5.	System Design	11–17
5.1	System Architecture	
5.2	Use Case Diagram	
6.	Implementation	15–17
6.1	Pseudocode	
7.	Testing	18–19
8.	Screenshots	20–26
	Future Enhancement and Conclusion	27
	References	28

	LIST OF FIGURES	
FIG. NO.	DESCRIPTION	PAGE NO.
5.1	System Architecture Diagram	12
5.2	Use Case Diagram	14
8.1	Overview / Dashboard Page	20
8.2	Home Page 2	21
8.3	Medical Services	21
8.4	Expert Doctors	22
8.5	Form	22
8.6	Patient Feedbacks	23
8.7	Contact Us	23
8.8	Appointments Booked	24
8.9	Jenkins dashboard	24
8.10	Github Repo Screenshot	24
8.11	Docker configurations	25

Chapter 1

INTRODUCTION

1.1 Background

With the increasing demand for fast and reliable software deployment, organizations are shifting from traditional deployment methods to automated DevOps workflows. Continuous Integration and Continuous Deployment (CI/CD) pipelines help developers automate software building, testing, and deployment processes.

Containerization technologies such as Docker provide lightweight and portable environments that ensure applications run consistently across different systems. Similarly, Jenkins enables automation of repetitive tasks, reducing manual effort and improving deployment reliability.

Healthcare and appointment booking systems require high availability, reliability, and efficient deployment mechanisms. Therefore, implementing a CI/CD pipeline for a Clinic Booking Website is an ideal use case to demonstrate modern DevOps practices and automation techniques.

This project combines web development and DevOps concepts to create a complete automated deployment solution.

1.2 Purpose

The purpose of this project is to design and implement an automated CI/CD pipeline for a Clinic Booking Website using GitHub, Docker, and Jenkins.

The project aims to:

- Automate the software deployment process
- Reduce manual intervention during deployment
- Improve deployment speed and consistency
- Demonstrate the practical implementation of DevOps tools

- Provide hands-on experience with CI/CD workflows and containerized applications

The project also helps in understanding how automation improves software quality, scalability, and operational efficiency in real-world development environments.

1.3 Objectives.

The main objectives of the project are:

1. To develop a responsive Clinic Booking Website.
2. To manage project source code using GitHub.
3. To implement version control and collaboration features.
4. To containerize the application using Docker.
5. To create a Docker image for consistent deployment.
6. To automate build and deployment using Jenkins.
7. To implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline.
8. To reduce deployment time and manual errors.
9. To improve application deployment reliability and scalability.
10. To gain practical knowledge of DevOps tools and workflows.

Chapter 2

LITERATURE REVIEW

A review of existing literature highlights the increasing adoption of web-based management systems, asynchronous backend frameworks, NoSQL databases, containerisation technologies, and CI/CD automation in modern software development and deployment environments.

2.1 Web-Based Event Management Systems — International Journal of Engineering Research (2019)

Proposed a web-based event management system for university campuses. The system enabled administrators to create and manage events using a centralised database. Users could register for events and view upcoming activities through a web portal. The application reduced dependence on physical notice boards and improved event discoverability. The system focused primarily on CRUD-based event operations.

Limitations:

- No DevOps integration or containerised deployment.
- No automated CI/CD pipeline support.
- Synchronous database operations caused performance bottlenecks under load.

2.2 Docker-Based Containerised Deployment — IEEE Research Journal (2020)

Proposed the use of Docker containers for deploying web applications in isolated environments. Applications and all their dependencies were packaged together inside containers, improving portability and consistency. Docker Compose was used to manage multi-container applications. The research focused on containerisation, environment isolation, and deployment automation.

Limitations:

- Did not include CI/CD pipeline integration.
- No health monitoring or orchestration-aware endpoint design.
- No Kubernetes or autoscaling support was demonstrated.

2.3 Continuous Integration and Continuous Deployment (CI/CD)

Shahin, Ali Babar, and Zhu (2017) conducted a systematic review on Continuous Integration, Delivery, and Deployment practices. The study explains that CI/CD pipelines automate software building, testing, and deployment processes, which improves software quality and reduces integration issues. The paper also discusses major CI/CD tools such as Jenkins, GitLab CI, and Travis CI.

Another review by Attipalli et al. (2021) emphasizes that CI/CD methodologies help in rapid and reliable software delivery by automating repetitive development tasks. The paper highlights Jenkins, GitHub Actions, and Docker as important technologies for modern cloud-native software deployment.

Soares et al. (2021) performed a systematic literature review on the effects of Continuous Integration in software development. Their research concluded that CI improves collaboration among developers, increases deployment frequency, and enhances software quality, although it may introduce infrastructure and configuration challenges

2.4 Kubernetes Horizontal Pod Autoscaling — Cloud Computing Journal (2022)

Studied the use of Kubernetes HPA for dynamically scaling containerised web applications based on CPU and memory metrics. The paper described the integration of liveness and readiness probes with Kubernetes deployments to ensure zero-downtime rolling updates. The server was used to expose resource utilisation data to the HPA controller.

Limitations:

- Did not provide a practical application-level implementation of probe endpoints.
- No integration with CI/CD pipelines was demonstrated.

2.5 Jenkins CI/CD with GitHub Webhook Integration — DevOps Quarterly (2021)

Investigated the use of Jenkins pipelines triggered by GitHub webhooks to implement automated build and test workflows. The paper described the use of ngrok and Cloudflare Tunnel to expose local Jenkins instances to the internet for webhook delivery.

Limitations: No Docker image build or Kubernetes deployment stages were included in the demonstrated pipeline.

Chapter 3

EXISTING SYSTEM AND PROPOSED SYSTEM

3.1 Existing System

In traditional software deployment environments, web applications are deployed manually by system administrators or developers. The deployment process usually involves manually copying files to the server, configuring the web server, restarting services, and verifying application functionality. This approach is time-consuming, error-prone, and difficult to maintain, especially when applications are updated frequently.

In many small-scale web applications, deployment processes lack automation and proper version control integration. Developers manually build and deploy applications after making changes, which increases the chances of configuration mismatches and deployment failures.

The traditional deployment model also faces several limitations such as:

- Manual deployment effort
- Increased deployment time
- High possibility of human errors
- Difficulty in maintaining consistency across environments
- Limited scalability
- Lack of automated monitoring and rollback mechanisms
- Delayed software delivery

In conventional systems, developers and operations teams often work separately, leading to communication gaps and inefficient workflows. Additionally, applications deployed directly on servers may behave differently in different environments due to dependency and configuration issues.

Therefore, there is a need for an automated, reliable, and scalable deployment system that can simplify software delivery and improve deployment efficiency.

6. Proposed System

The proposed system implements an automated Continuous Integration and Continuous Deployment (CI/CD) pipeline for the Clinic Booking Website using GitHub, Docker, and Jenkins.

In this system, GitHub is used as the source code repository, Docker is used for application containerization, and Jenkins automates the build and deployment process. Whenever changes are pushed to the GitHub repository, Jenkins automatically detects the updates, pulls the latest code, builds a Docker image, and deploys the application container.

The proposed system eliminates manual deployment tasks and ensures faster, more reliable software delivery.

Working of the Proposed System

1. The developer pushes updated code to GitHub.
2. Jenkins continuously monitors the GitHub repository.
3. Jenkins automatically pulls the latest source code.
4. Docker builds a new container image for the application.
5. Existing containers are stopped and removed.
6. A new container is deployed automatically.
7. The updated Clinic Booking Website becomes accessible to users.

Chapter 4

SYSTEM REQUIREMENTS

4.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i5 (8th Gen) or equivalent
RAM	8 GB (16 GB recommended for running Docker + Kubernetes locally)
Storage	20 GB free disk space
Network	Stable internet connection (for GitHub webhook and Docker image pulls)

4.2 Software Requirements

- Backend

Technology	Version
Python	3.11
FastAPI	0.115.0
Uvicorn (dev)	0.30.6
Gunicorn + UvicornWorker (prod)	22.0.0
Motor (async MongoDB driver)	3.5.1
PyMongo	4.8.0
Pydantic v2	2.8.2
Jinja2	3.1.2
python-dotenv	1.0.1
python-multipart	0.0.9

• Frontend

Technology	Detail
HTML5	Jinja2 server-side rendered templates
CSS	Vanilla CSS with custom design system
JavaScript	Vanilla ES6+ (no frameworks)
Icons	Lucide Icons (CDN, SVG-based)
Typography	Inter (Google Fonts)

• Database

Technology	Version
MongoDB	7.0

• DevOps & Infrastructure

Tool	Role
Docker	Application and database containerisation
Docker Compose	Local multi-container orchestration
Jenkins	CI/CD pipeline automation
GitHub	Source control and webhook trigger
ngrok / Cloudflare Tunnel	Exposes local Jenkins to the internet
Kubernetes	Production orchestration with HPA support
pytest	Automated test runner
httpx	Async HTTP client for API testing
pytest-asyncio	Async test support

Functional Requirements

- User Authentication: The system allows users to access and interact with the clinic booking website.
- Appointment Booking: Users can book clinic appointments through the web interface.
- GitHub Integration: The application source code is managed and stored using GitHub.
- Automated Build Process: Jenkins automatically builds the application after code updates.
- Automated Deployment: The CI/CD pipeline deploys the latest application automatically.
- Docker Containerization: The application runs inside Docker containers for consistent deployment.
- Web Hosting Support: Nginx is used to host and serve the web application.
- Continuous Integration: Jenkins continuously monitors and integrates code changes from GitHub.

Non-Functional Requirements

- Performance: The CI/CD pipeline enables faster build and deployment processes.
- Reliability: Docker containers ensure consistent application behavior across environments.
- Scalability: The containerized application can be scaled easily when required.
- Availability: The website remains accessible with minimal deployment downtime.
- Maintainability: GitHub and Jenkins simplify project maintenance and updates.
- Security: Controlled access to GitHub and Jenkins improves deployment security.
- Portability: Docker allows the application to run on multiple platforms consistently.
- Usability: The website provides a responsive and user-friendly interface.

Chapter 5

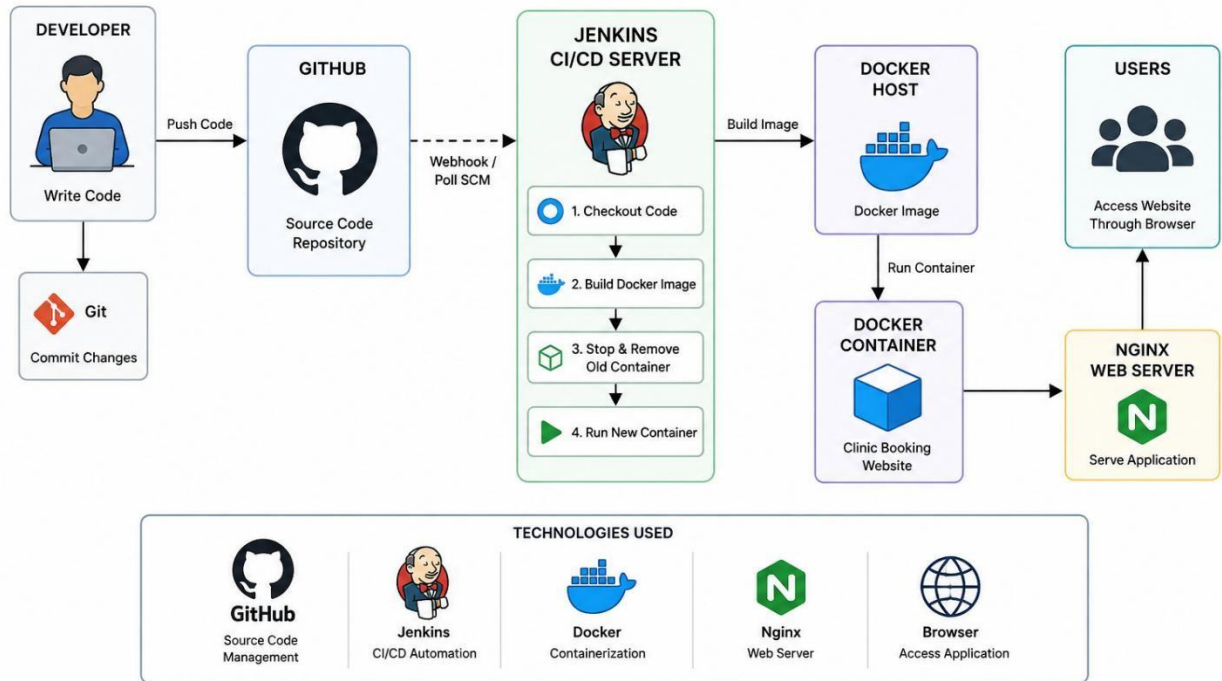
SYSTEM DESIGN

5.1 System Architecture

- The system architecture of the Clinic Booking Website is designed using DevOps principles and CI/CD automation tools. The architecture integrates GitHub, Jenkins, Docker, and Nginx to automate the application deployment process.
- The workflow begins when the developer writes and pushes the source code to the GitHub repository. GitHub acts as the centralized version control system that stores and manages the project files.
- Jenkins continuously monitors the GitHub repository using webhooks or polling mechanisms. Whenever new code changes are detected, Jenkins automatically triggers the CI/CD pipeline. The pipeline performs several tasks such as cloning the repository, building the Docker image, stopping the old container, and deploying a new container.
- Docker is used to containerize the Clinic Booking Website. The application files are packaged into a Docker image, ensuring consistent deployment across different environments. The Docker container runs the application in an isolated and portable environment.
- Nginx acts as the web server inside the Docker container and serves the Clinic Booking Website to end users through a web browser. Users can access the application and interact with the appointment booking system seamlessly.
- The architecture improves deployment speed, scalability, consistency, and reliability while reducing manual intervention and deployment errors.

System Architecture

Clinic Booking Website CI/CD Pipeline



System Architecture Diagram — Fig 5.1

5.2 Use Case Diagram

The primary actors and use cases in the EventOps system are:

The Use Case Diagram represents the interaction between users and the Clinic Booking Website system.

It identifies the actors and the functionalities provided by the system.

Actors

- User
- Developer
- Jenkins Server

Use Cases

User

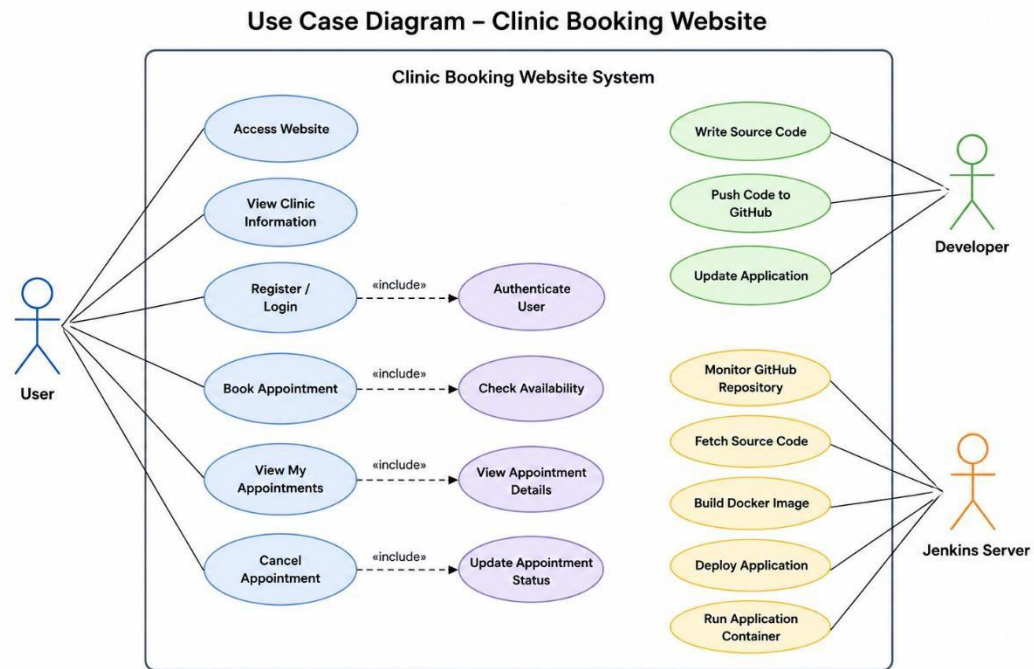
- Access Website
- View Clinic Information
- Book Appointment
- Navigate Website

Developer

- Write Source Code
- Push Code to GitHub
- Update Application

Jenkins Server

- Fetch Source Code
- Build Docker Image
- Deploy Application
- Monitor Repository



Use Case Diagram — Fig 5.2

Chapter 6

IMPLEMENTATION

Pseudocode

CI/CD Pipeline Module

START

Developer writes application code

Push source code to GitHub repository

Jenkins monitors GitHub repository

IF new changes are detected THEN

 Clone latest source code

 Build Docker image

 Stop existing Docker container

 Remove old Docker container

 Run new Docker container

 Deploy application using Nginx

END IF

User accesses Clinic Booking Website through browser

END

➤ Docker Build Module

START

Read Dockerfile

Pull Nginx base image

Copy application files into Docker container

Build Docker image

Expose application port

Store Docker image locally

END

➤ **Jenkins Deployment Module**

START

Trigger Jenkins pipeline

Fetch latest code from GitHub

Execute Docker build command

IF existing container is running THEN

 Stop running container

 Remove old container

END IF

Run new Docker container

Verify deployment status

END

➤ **User Appointment Booking Module**

START

User opens Clinic Booking Website

User enters appointment details

Validate input fields

IF validation fails THEN

 Display error message

ELSE

 Save appointment details

 Display booking confirmation

END IF

END

Chapter 7

TESTING

7. Testing

Testing is performed to verify that the Clinic Booking Website and CI/CD pipeline work correctly and efficiently. The testing process ensures that the application functions as expected without errors during deployment and execution.

7.2 Types of Testing

➤ Functional Testing

Functional testing verifies whether all features of the application work correctly. **Tested**

Features

- Website loading
- Navigation functionality
- Appointment booking interface
- Responsive design
- CI/CD pipeline execution

➤ Integration Testing

Integration testing checks the communication between GitHub, Jenkins, Docker, and Nginx. **Tested**

Integrations

- GitHub with Jenkins
- Jenkins with Docker
- Docker with Nginx
- End-to-end deployment workflow

➤ Deployment Testing

Deployment testing verifies whether the application deploys successfully using Docker containers.

Deployment Checks

- Docker image creation
- Container startup
- Port mapping
- Nginx server execution

➤ Performance Testing

Performance testing ensures the application responds efficiently during usage. **Performance Parameters**

- Website loading speed
- Container startup time
- Deployment execution time
- Browser response time

7.3 Test Cases

Test Case	Expected Result	Status
Push code to GitHub	Jenkins pipeline triggered	Passed
Build Docker image	Image created successfully	Passed
Deploy Docker container	Container runs successfully	Passed
Access website in browser	Website loads correctly	Passed
Appointment interface check	Form displays properly	Passed
Jenkins integration	Pipeline executes without errors	Passed

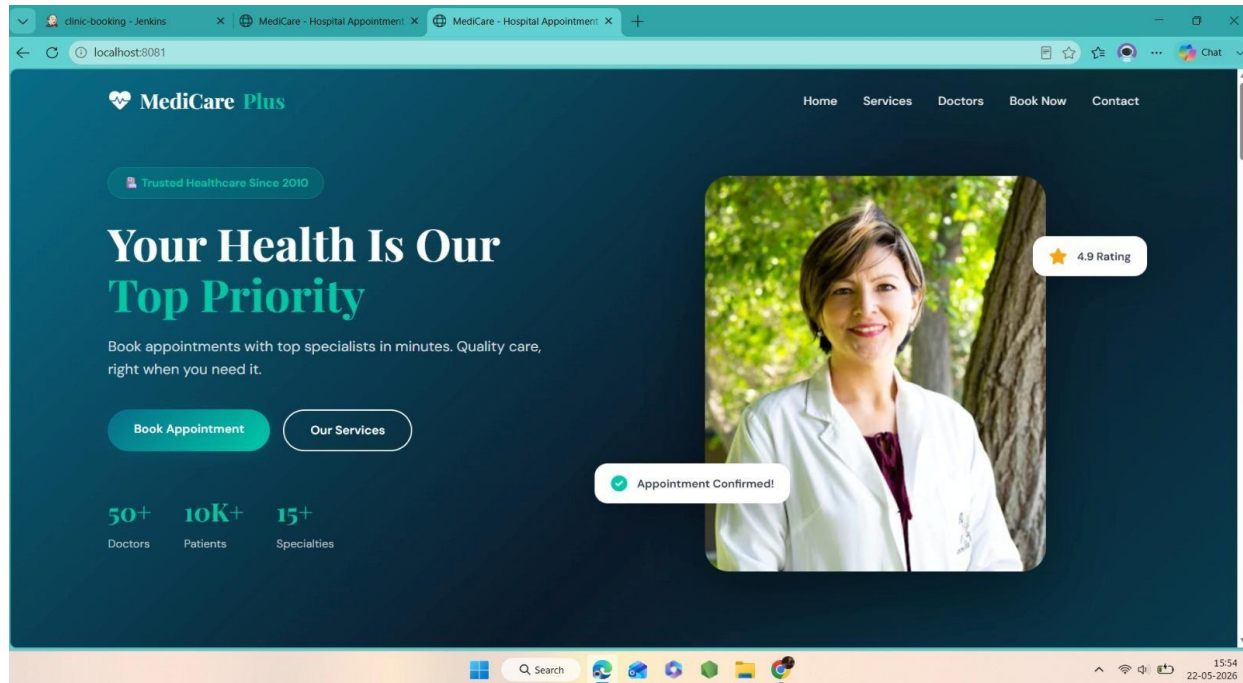
7.4 Testing Tools Used

- Jenkins
- Docker
- GitHub
- Web Browser
- Linux Terminal

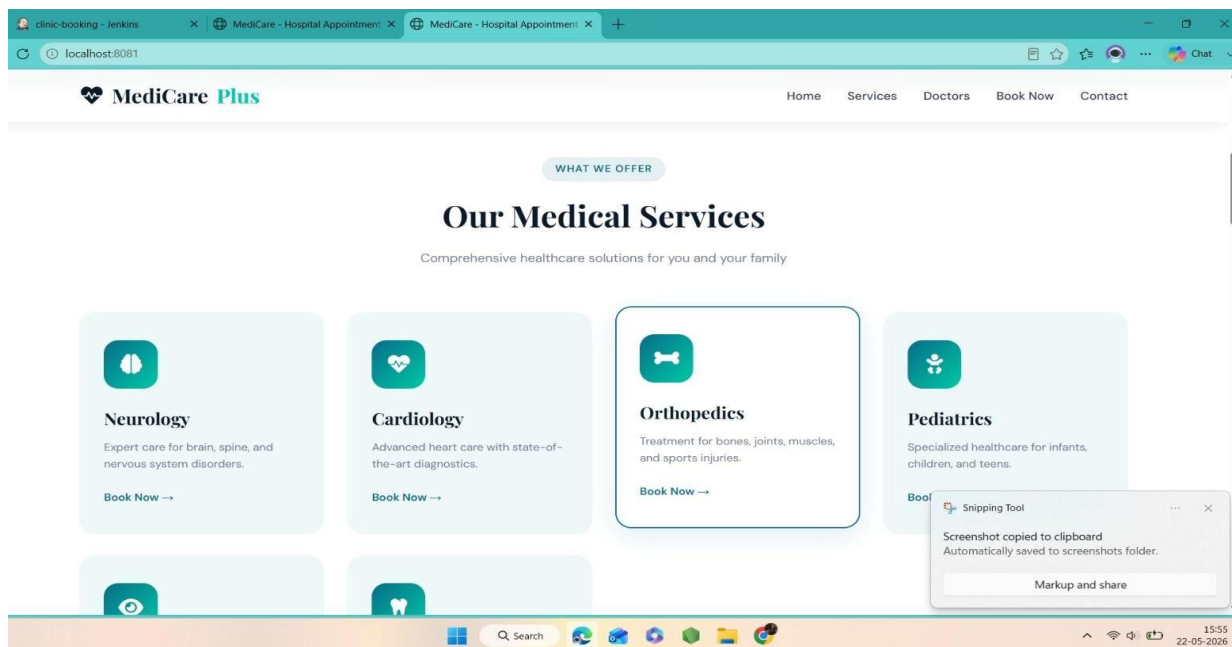
10.5 Testing Outcome

The testing process confirmed that the Clinic Booking Website and CI/CD pipeline function correctly. The application was successfully deployed using Docker and Jenkins automation without deployment failures.

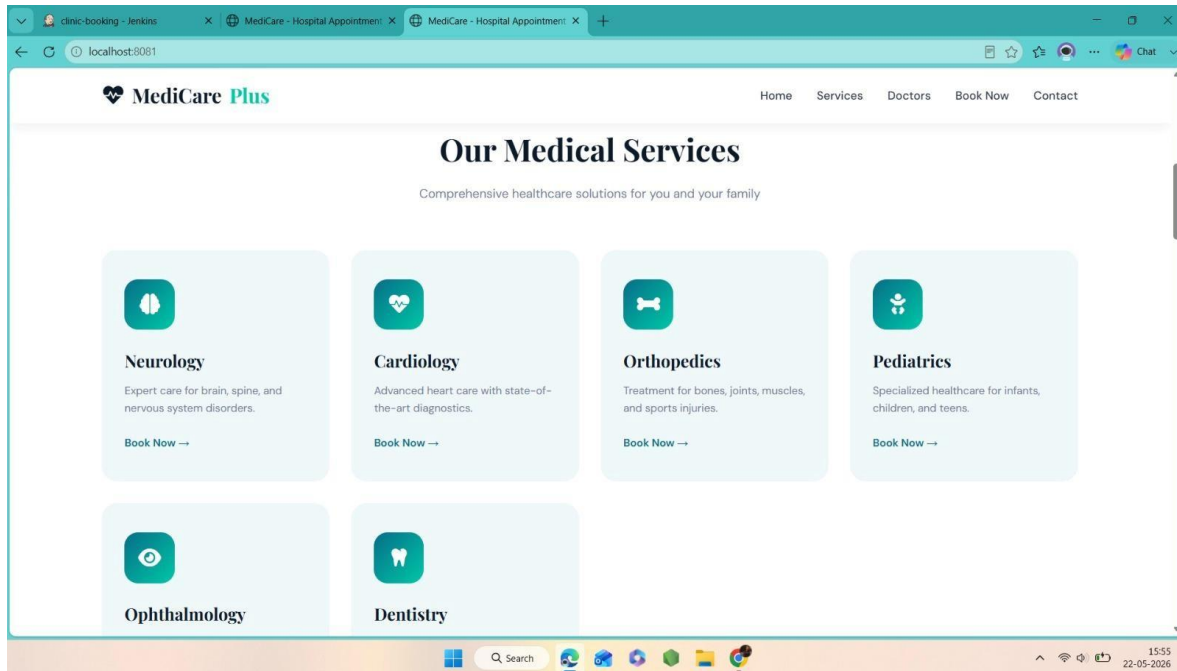
Chapter 8



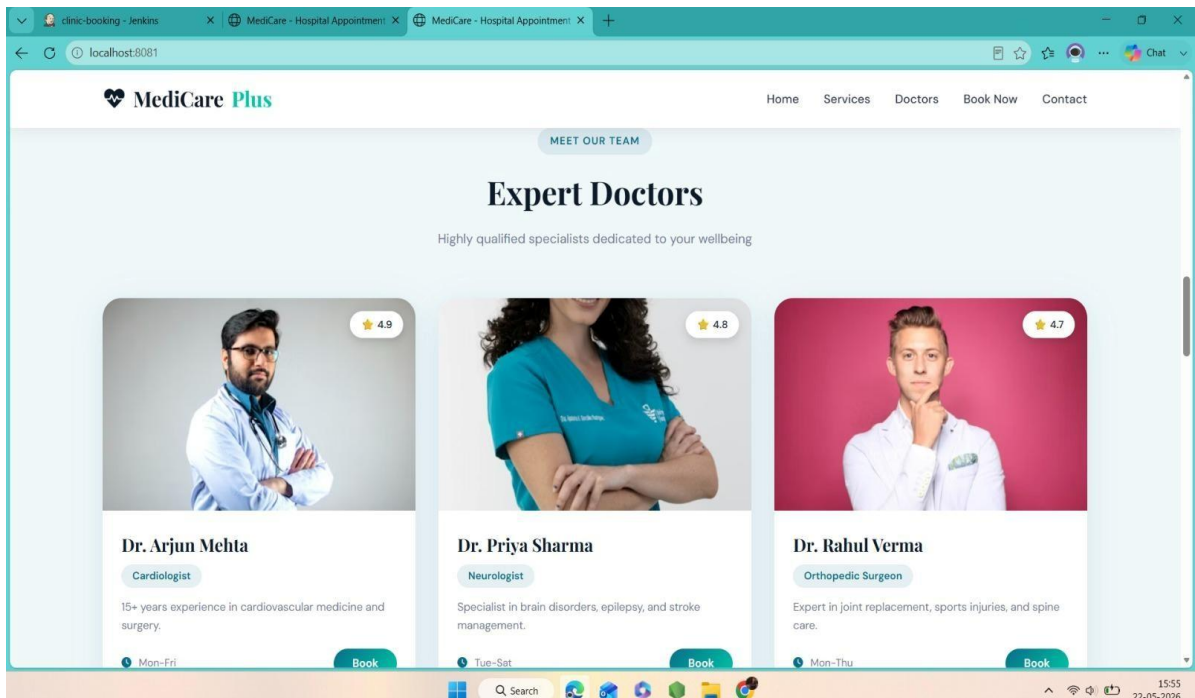
Screenshot 8.1 — home page



Screenshot 8.2 — Home page 2



Screenshot 8.3 — Mdeical Services



Screenshot 8.4— Doctors dashboard

MediCare Plus

Home Services Doctors Book Now Contact

Why Choose Us?

- ✓ Instant appointment confirmation
- ✓ No waiting in long queues
- ✓ 50+ specialist doctors
- ✓ Online & in-person consultations
- ✓ 24/7 emergency support
- ✓ Insurance accepted

Emergency?
Call us immediately
+1 (800) 911-CARE

Full Name
Enter your name

Phone Number
+1 (555) 000-0000

Email Address
your@email.com

Age
Your age

Department
Select Department

Doctor
Select Doctor

Appointment Date
dd-mm-yyyy

Time Slot
Select Time

Symptoms / Notes
Describe your symptoms or reason for visit...

Confirm Appointment

Screenshot 8.5 — form

MediCare Plus

Home Services Doctors Book Now Contact

GET IN TOUCH

Contact Us

We're here to help 24/7

Location
456 Health Avenue, Medical District,
New York, NY 10001

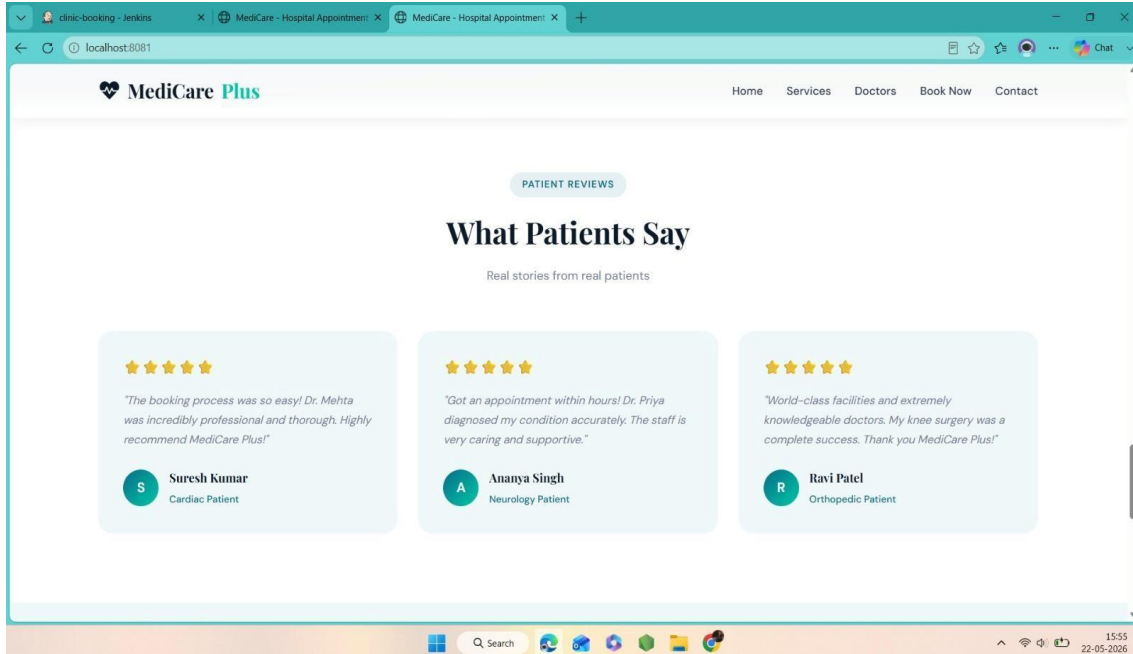
Phone
+1 (800) 911-CARE
+1 (555) 234-5678

Email
care@medicareplus.com
appointments@medicareplus.com

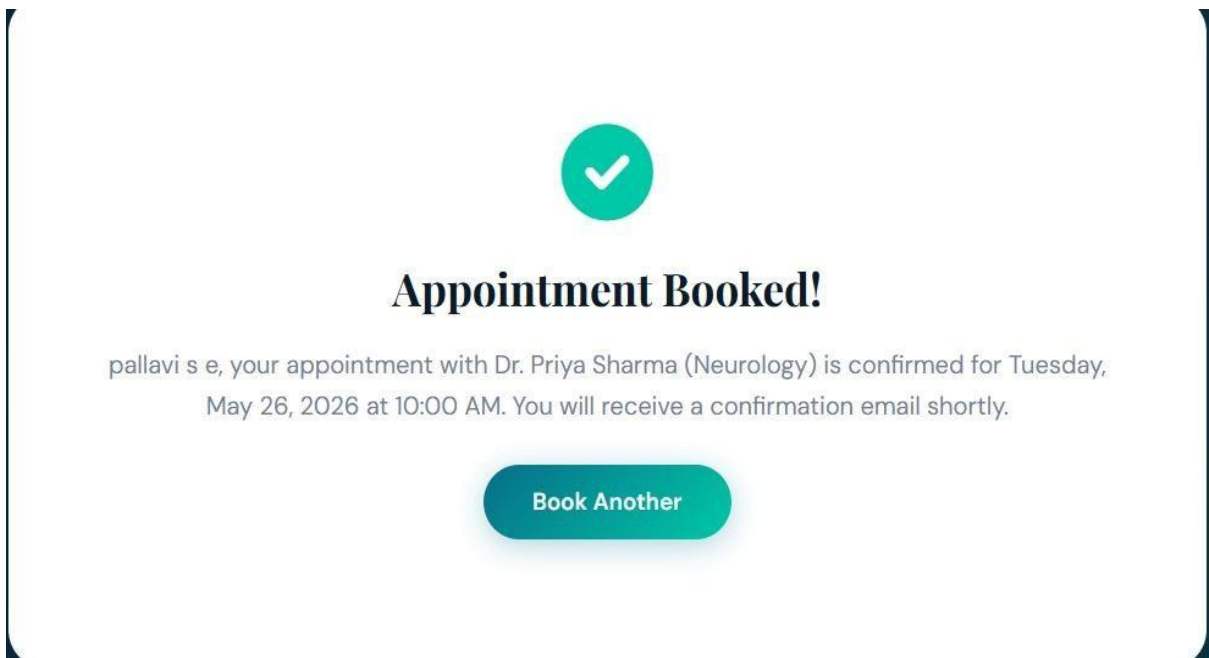
Hours
Mon-Sat: 8AM - 8PM
Emergency: 24/7

MediCare Plus
Your health, our mission.

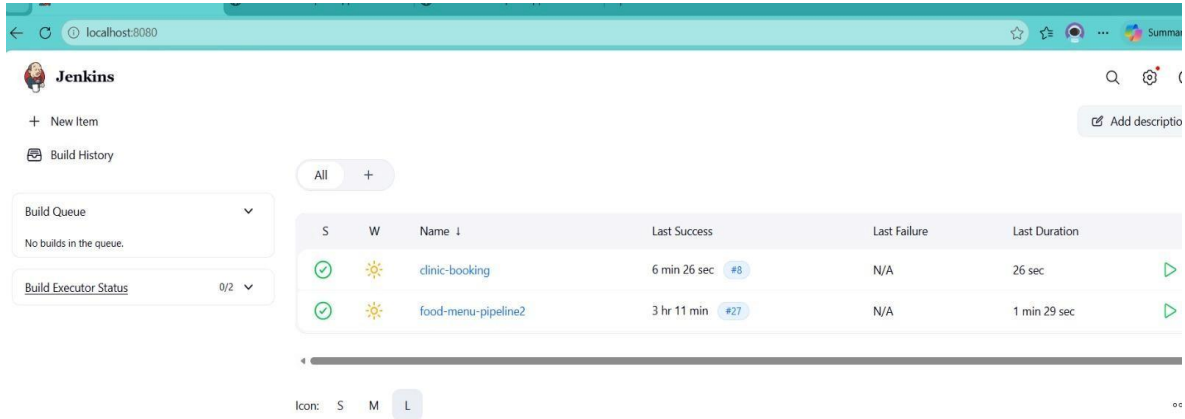
Screenshot 8.7 — Contact page



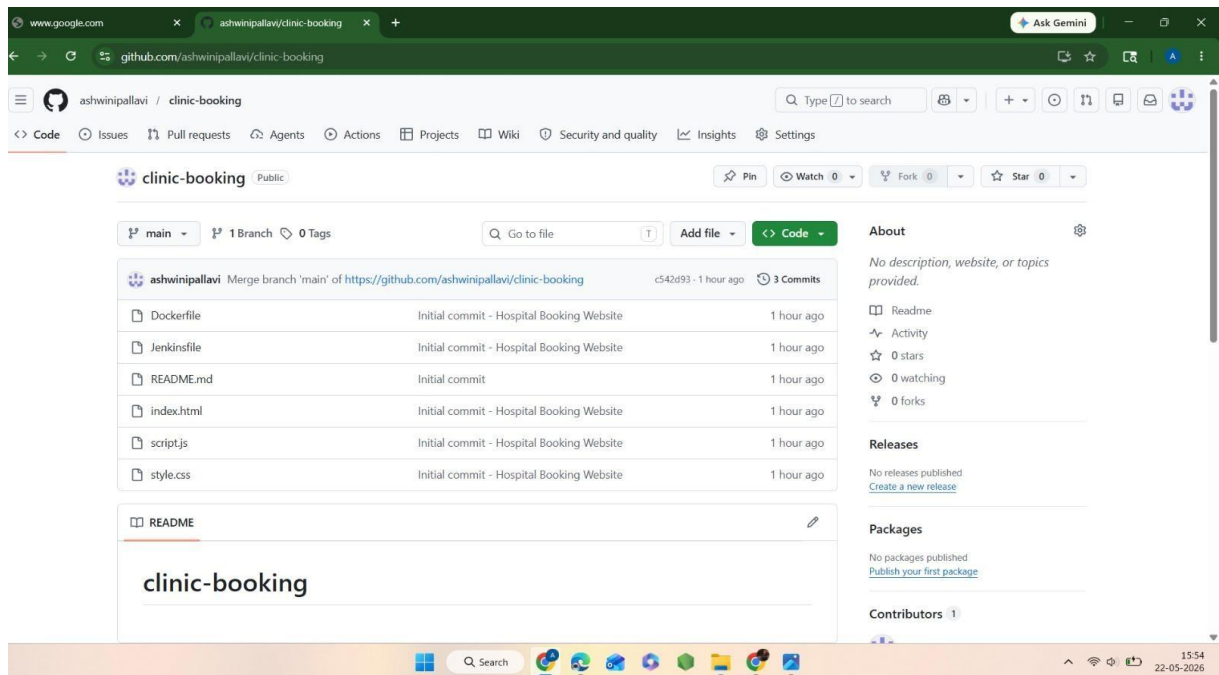
Screenshot 8.16— Feedback page



Screenshot 8.8— Appointment Confirmation Page



Screenshot 8.9- Jenkins

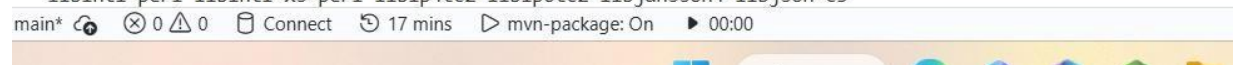


Screenshot 8.10— GitHub Repository and Webhook Configuration Page

```

PS C:\Users\User\clinic-booking> git push -u origin main
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 8 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 9.09 KiB | 1.51 MiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/ashwinipallavi/clinic-booking.git
   b89d9a2..c542d93  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\User\clinic-booking> docker start jenkins
Error response from daemon: No such container: jenkins
Error: failed to start containers: jenkins
PS C:\Users\User\clinic-booking> docker run -d -p 8080:8080 -p 50000:50000 --name jenkins -v jenkins_home:/
ed73c79711d90f4bcbf4ae9f1cf34d21301f6212b4f2b87588e1d584baee6abb
PS C:\Users\User\clinic-booking> docker exec -u root jenkins apt-get update
Get:1 http://deb.debian.org/debian trixie InRelease [140 kB]
Get:2 http://deb.debian.org/debian trixie-updates InRelease [47.3 kB]
Get:3 http://deb.debian.org/debian-security trixie-security InRelease [43.4 kB]
Get:4 http://deb.debian.org/debian trixie/main amd64 Packages [9671 kB]
Get:5 http://deb.debian.org/debian trixie-updates/main amd64 Packages [5412 B]
Get:6 http://deb.debian.org/debian-security trixie-security/main amd64 Packages [166 kB]
Fetched 10.1 MB in 4s (2245 kB/s)
Reading package lists...
PS C:\Users\User\clinic-booking> docker exec -u root jenkins apt-get install -y docker.io
Reading package lists...
Building dependency tree...
Reading state information...
The following additional packages will be installed:
  apparmor containerd criu dbus dbus-bin dbus-daemon dbus-session-bus-common
  dbus-system-bus-common dbus-user-session dmsetup docker-buildx docker-cli
  gettext-base iproute2 iptables libapparmor1 libbpf1 libcap2 libcap2-bin
  libcompel1 libcryptsetup12 libdbus-1-3 libdevmapper1.02.1 libelf1t64
  libintl-perl libintl-xs-perl libip4tc2 libip6tc2 libjansson4 libjson-c5

```



Screenshot 8.11— Docker

CONCLUSION AND FUTURE ENHANCEMENTS

Conclusion:

The integration of Docker ensured consistent and isolated deployment of both the application and the database, while Docker Compose simplified local multi-container orchestration with healthcheck-based dependency management. Jenkins automated the Continuous Integration workflow by executing the pytest test suite on every GitHub push, preventing untested code from reaching deployment.

The Clinic Booking Website project successfully implemented a Continuous Integration and Continuous Deployment (CI/CD) pipeline using GitHub, Docker, and Jenkins. The project demonstrated how DevOps tools and automation techniques improve software deployment efficiency, reliability, and consistency.

Future Enhancements:

The project can be further improved by integrating a backend framework and database support for storing appointment details dynamically. Additional features such as user authentication, online payment integration, email notifications, and admin dashboards can be implemented to enhance functionality. The deployment process can also be extended using Kubernetes for container orchestration and cloud platforms such as AWS or Azure for scalable hosting. Security mechanisms, automated testing, monitoring tools, and mobile application support can also be added in future versions of the system.

Monitoring and Observability Stack A Prometheus metrics endpoint (`/metrics`) can be added to expose request rates, latency percentiles, and error rates. A Grafana dashboard can be configured to visualise these metrics, and container logs can be shipped to a centralised system such as Grafana Loki or the ELK stack.

Cloud Deployment The application can be deployed on a managed Kubernetes service such as AWS EKS, Google GKE, or Azure AKS to demonstrate production-level availability, auto-scaling, and geographic distribution with a TLS-enabled NGINX Ingress Controller.

REFERENCES

1. [FastAPI Documentation, "FastAPI — Modern, Fast Web Framework for Building APIs with Python," Available: https://fastapi.tiangolo.com](https://fastapi.tiangolo.com)
 2. Docker Documentation, "Docker Containerization Platform," <https://www.docker.com>
 3. Jenkins Documentation, "Jenkins Automation Server," Available: <https://www.jenkins.io>
 4. MongoDB Documentation, "MongoDB 7.0 — NoSQL Database," Available: <https://www.mongodb.com/docs>
 5. Motor Documentation, "Motor — Async Python Driver for MongoDB," Available: <https://motor.readthedocs.io>
 6. Pydantic Documentation, "Pydantic v2 — Data Validation for Python," Available: <https://docs.pydantic.dev>
 7. Kubernetes Documentation, "Kubernetes — Production-Grade Container Orchestration," Available: <https://kubernetes.io/docs>
 8. GitHub Documentation, "GitHub Webhooks," Available: <https://docs.github.com/en/webhooks>
 9. pytest Documentation, "Python Testing Framework," Available: <https://docs.pytest.org>
 10. K. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, 2014.
 11. M. Fowler, "Continuous Integration and Continuous Deployment," ThoughtWorks Publications, 2013.
 12. International Journal of Engineering Research, "Web-Based Event Management Systems for Educational Institutions," Vol. 8, No. 3, 2019.
-

